

Historical Conversational AI System

Cold-Path Ingestion & Bronze Landing Zone

Albert Puiggròs Edwin Santiago Delgado Jenny Müller

https://github.com/apuiggros/BDM_PROJECT

April 12, 2026

Abstract

This report documents the design and implementation of the first deliverable (P1) of a Big Data Management pipeline for a **Historical Conversational AI system** — a multimodal content generation platform capable of producing interview-style interactions between historical figures and today’s news. The system is designed to support multiple output formats, from text-based Q&A to short-form video content for platforms such as TikTok or Instagram Reels, with the underlying data architecture remaining agnostic to the chosen modality.

P1 delivers a fully operational **Bronze Layer Data Lakehouse** ingesting data across seven external sources — the Philosophers API, Project Gutenberg (70,000+ public-domain texts), the Wikipedia REST API, iTunes podcast RSS feeds, the GNews top-headlines API, the Wikiquote MediaWiki API, and the Philosophy Stack Exchange — covering nine historical figures across three domains (philosophy, science, and literature). Data collection spans all three required categories: structured biographical metadata, semi-structured JSON, and unstructured content. To satisfy the volume requirements of a Big Data pipeline, the system ingests an initial footprint of **approximately 600 MB of plain-text philosophical and scientific works, over 2 GB (40+ hours) of podcast audio, and daily news snapshots growing at ~150 KB/day**. The cold-path batch pipeline is orchestrated daily by **Apache Airflow** across seven parallel ingestion tasks, while an **Apache Kafka** hot-path pipeline continuously captures simulated real-time character mention events. All structured outputs are written as **Delta Lake** tables on a **MinIO** S3-compatible object store, providing ACID guarantees, time travel, and Parquet columnar storage. The entire eight-container infrastructure is containerized via **Docker Compose** and reproducible from a cold start with a single command.

1. Contextualization

1.1 Domain and Business Value

The domain of this project sits at the intersection of **Digital Humanities, Knowledge Engineering, and Generative AI**. The central challenge is to power a **multimodal conversational AI system** capable of generating realistic, interview-style content featuring famous historical figures — beginning with classical philosophers, scientists, and literary icons.

The core question this system is designed to answer is deliberately provocative: “*How would Friedrich Nietzsche react to today’s headlines on artificial intelligence?*” Answering this credibly requires a system that simultaneously knows the philosopher’s own vocabulary and reasoning style, the conventions of spoken interview dynamics, and the current state of the world — a combination no single off-the-shelf dataset provides.

Value Proposition

A production-grade data lakehouse that assembles six orthogonal knowledge pillars — biographical facts, canonical primary texts, verified quotes, community Q&A examples, conversational audio dynamics, and daily news awareness — into a unified Bronze Layer from which a future AI content generation pipeline can be built.

The system addresses a real gap in the AI landscape: while large language models can approximate the “style” of a historical figure from pretraining data, they cannot reliably ground that persona in *primary sources* (the figure’s actual writings), *real-time context* (today’s news), or *acoustic performance* (the prosodic rhythm of spoken conversation). This pipeline provides the raw data infrastructure to bridge all three gaps.

1.2 End-Goal and Consumption Model

The precise consumption format of the system is intentionally left flexible at this stage, as the architecture is designed to support multiple output modalities without changes to the underlying data pipeline. Three progressive delivery formats are under consideration, ordered by technical complexity:

- **Q&A Engine:** The most immediate application is a text-based question-and-answer interface where users submit prompts (e.g. “*What would Kant think of social media?*”) and receive responses grounded in the figure’s primary texts and today’s news. This serves as the core reasoning layer and the foundation for all richer formats above it. The Philosophy Stack Exchange data ingested in P1 provides a direct training signal for this format: real humans have asked and answered thousands of philosophy questions in exactly the Q&A register the system needs to replicate.
- **Short-form video content (TikTok / Reels):** The primary commercial target. Using the Q&A engine as a script generator, short clips (30–90 seconds) are automatically produced featuring a synthetic voice and AI-generated avatar of the historical figure reacting to a trending news headline. These clips can be posted programmatically via platform APIs, enabling a fully automated content pipeline from news ingestion to published video. The Wikiquote verified quotes ingested in P1 provide a grounding layer that prevents the AI from

fabricating statements — the system can anchor each clip in something the figure actually said before extrapolating to the modern context.

- **Long-form podcast episodes:** A premium format in which a full interview structure is generated — multiple questions, follow-up exchanges, thematic coherence — and rendered as an audio episode. This requires richer context retrieval and a more sophisticated dialogue management layer, and is considered a longer-term milestone.

The short-form video format is the most commercially attractive in the near term: production cost is near-zero once the pipeline is operational, content is highly shareable, and the novelty of a historical figure reacting to today’s headlines has strong viral potential. Monetization pathways include platform ad revenue, branded content partnerships (e.g. an education company sponsoring a “*Plato reacts to AI*” series), and licensing the generation system itself as a SaaS product to media companies or EdTech platforms.

The data architecture designed in P1 is agnostic to which of these formats is ultimately prioritized. The same Bronze Layer — primary texts, biographical facts, conversational audio, daily news, verified quotes, and curated Q&A pairs — feeds all three consumption models. The choice of output format is a P2 Exploitation Zone decision, not an ingestion one.

Source	Format	Data Category	Path	Knowledge Pillar
Philosophers API	REST/JSON	Semi-structured	Cold	Biographical facts & portraits
Project Gutenberg	REST/plain text	Unstructured	Cold	Primary writings & vocabulary
Wikipedia REST API	REST/JSON	Semi-structured	Cold	Universal biographical backbone
iTunes RSS + Audio	RSS / MP3 binary	Unstructured	Cold	Conversational dynamics
GNews API	REST/JSON	Semi-structured	Cold	Current events awareness
Wikiquote API	MediaWiki/JSON	Semi-structured	Cold	Verified quotes per figure
Philosophy Stack Exchange	REST/JSON	Semi-structured	Cold	Curated Q&A format examples
Simulated Kafka Stream	JSON events	Semi-structured	Hot	Real-time character mentions

1.3 Data Sources

Seven external data sources are integrated, covering all three required data categories (structured, semi-structured, and unstructured) across two ingestion paths. Each source was selected to fill a specific gap in the AI system’s knowledge requirements, from biographical grounding and

primary-text vocabulary to verified quote attribution and real-world conversational dynamics.

1.3.1 Philosophers API (philosophersapi.com)

A public, authentication-free REST API exposing a catalog of 114 philosopher records. Each record is a rich JSON object containing biographical metadata (school of thought, era, notable concepts), academic deep-links (Stanford Encyclopedia of Philosophy, Internet Encyclopedia of Philosophy, Wikipedia), and a nested `images` dictionary with portrait thumbnails, full illustrations, and face crops. The full catalog is returned in a single flat list with no pagination. This source provides **structured ground truth** for all target philosophy figures.

Notably, each ingested record already contains a `speLink` field pointing to the figure’s dedicated article on the Stanford Encyclopedia of Philosophy (SEP) — one of the most authoritative and structured secondary sources in academic philosophy. These URLs are extracted and stored in the Bronze Layer under the `_academic_links.stanford_sep` field of the philosophers Delta table. While active scraping of SEP content is outside the scope of P1, these stored URLs represent a direct upgrade path for P2: a future `sep_ingest.py` script could retrieve the full article text for each figure, providing structured, topic-organized secondary analysis that is significantly more RAG-friendly than raw Gutenberg prose. Alternatively, the URLs can be surfaced directly to the AI system at inference time, instructing it to consult the SEP article as an authoritative reference when answering questions on specific philosophical topics.

1.3.2 Project Gutenberg via Gutendex (gutendex.com)

Gutendex is a community-built REST API wrapping Gutenberg’s catalog of over 70,000 public-domain texts. It supports author-based search and returns structured book metadata including multi-format download URLs. The plain-text downloads themselves are **unstructured** data — raw literary content with Gutenberg boilerplate headers and footers that will be stripped in the Trusted Zone (P2). The nine target figures yield over 150 downloadable texts, establishing a concrete unstructured data volume of **approximately 600 MB of raw prose**.

1.3.3 Wikipedia REST API

The Wikipedia summary endpoint (`/api/rest_v1/page/summary/{title}`) returns a concise JSON biography for any article title, including a plain-text extract, page image metadata, and normalized structured fields. No API key is required. This source acts as a **universal backbone** — every figure in the registry automatically receives a factual biography regardless of domain, making it the most robust fallback when other sources have coverage gaps.

1.3.4 iTunes RSS + Podcast Audio

Podcast audio is ingested via a **discovery-based approach**: the iTunes Search API is queried with broad topic terms (e.g. “philosophy”, “history of ideas”) to find the top-ranked channels by popularity. For each discovered channel, the RSS feed is parsed to extract the latest episode enclosures (MP3/M4A URLs), which are then streamed directly into MinIO without buffering to disk. This provides **unstructured audio** — the acoustic raw material for future voice modelling and tone analysis. This discovery approach pulls down roughly **40 hours of conversational audio per run (totalling ~2 GB of binary MP3 data)**. A JSON metadata envelope is generated per episode for provenance tracking.

1.3.5 GNews API — Top Headlines

The GNews `/api/v4/top-headlines` endpoint returns the top trending stories from Google News across three categories: *world*, *technology*, and *science*. The design choice is intentional: rather than searching for philosophy-related topics, the pipeline captures **the current news cycle** — the raw set of ideas and events that define the current moment. This is what the AI uses to simulate real-time reactions. The free tier allows 100 requests/day, making daily batch ingestion well within quota. The resulting daily payload generates a snapshot of **~150 KB**, **steadily growing the Delta Lake history by roughly 4.5 MB per month**.

1.3.6 Wikiquote API — Verified Quotes

The Wikiquote MediaWiki API (en.wikiquote.org/w/api.php) returns structured page content for any historical figure, including their verified and sourced quotations. This source addresses a critical risk in the AI generation pipeline: without grounding, a language model may plausibly fabricate quotes and attribute them to a historical figure. The Wikiquote data provides a verified quotation index per figure that can be used both as a RAG retrieval source and as a hard constraint at inference time — ensuring the system only attributes statements that the figure actually made or that are clearly derivable from their verified writings. No API key is required. An idempotency pre-check is applied per figure since quotes rarely change, avoiding unnecessary refetching.

Storage: `s3://landing-zone/wikiquote/raw_json/{domain}/{slug}_wikiquote.json`

1.3.7 Philosophy Stack Exchange — Q&A Format Examples

The Stack Exchange API (api.stackexchange.com) exposes the top-voted questions and answers from the philosophy-specific Stack Exchange community. The script fetches the 500 highest-voted questions (five pages of 100), tagged with an ingestion timestamp, and stores them as a daily overwrite snapshot. This source serves a fundamentally different purpose from the others: it is not about the historical figures themselves, but about **the format the AI must learn to replicate**. Real humans have formulated philosophy questions and expert answers in exactly the conversational register the system needs to generate — making this dataset a direct training and evaluation signal for the Q&A output layer. A one-second delay between page requests respects the Stack Exchange API rate limit.

Storage: `s3://landing-zone/philosophy_se/raw_json/philosophy_se_snapshot_{YYYYMMDD}.json`

1.3.8 Simulated Kafka Stream — Character Mentions

To fulfil the streaming requirement, a Kafka producer continuously generates synthetic **real-time mention events** for all registered characters — simulating a live social media or news ticker feed. Each event carries a character name, domain, message text, sentiment score, and source platform. While the data is currently synthetic, the infrastructure is entirely production-compatible. In a real-world deployment, this simulated producer would be seamlessly replaced by a live **Twitter (X) or Reddit API listener**. This hot path captures time-sensitive social signals, enabling P2 downstream applications to perform real-time sentiment monitoring of the figures, track emerging trending topics before they hit the daily news cycle, and allow the AI to react instantly to viral discourse.

1.4 Target Entities

The pipeline currently targets **9 historical figures** across three domains, all managed through a single registry file (`character_registry.py`):

Figure	Domain	Rationale
Plato	Philosophy	Socratic method translates perfectly to a multi-turn Q&A format; ideal for questioning modern societal structures.
René Descartes	Philosophy	Rigorous skepticism provides a strong, analytical persona for dissecting modern “post-truth” news and deepfakes.
Immanuel Kant	Philosophy	Highly structured reasoning offers rigid, predictable moral reactions to modern AI and tech ethics headlines.
G.W.F. Hegel	Philosophy	Dialectical approach allows for nuanced interview responses that synthesize opposing viewpoints in today’s polarized news.
Friedrich Nietzsche	Philosophy	Provocative, aphoristic style guarantees highly engaging, controversial soundbites perfectly suited for short-form video.
Albert Einstein	Science	Use of thought experiments and accessible explanations maps well to reacting to modern scientific and technological breakthroughs.
Charles Darwin	Science	Methodical observation and inference style maps naturally to a structured, evidence-based Q&A format.
Oscar Wilde	Literature	Wit and social critique; excellent conversational style and comedic timing for media consumption.
Mark Twain	Literature	Sharp American satirical commentary provides highly shareable, humorous reactions to political and cultural news.

Adding a new figure to the entire pipeline requires editing exactly **one file** — `ingestion/character_registry.py` — after which all ingestion scripts automatically include the new entity on the next run.

2. Architecture Design

2.1 High-Level Overview

The pipeline follows a **Registry-Driven, Dual-Path Micro-Ingestion architecture** built on the **Medallion pattern** (Bronze / Silver / Gold layers). P1 implements the full Bronze Layer. The Silver and Gold Layers are architected as black boxes here, to be instantiated in P2.

The architecture is organized around three fundamental separations:

- **Separation of paths:** Cold (batch) and Hot (streaming) data travel through independent infrastructures, meeting only at the Landing Zone.
- **Separation of concerns:** Ingestion scripts are stateless and source-specific; the orchestrator handles scheduling, retries, and dependency management; MinIO handles storage; Delta Lake handles metadata.
- **Separation of layers:** Raw data (Bronze), cleaned data (Silver, P2), and curated data (Gold, P2) are physically isolated in distinct bucket prefixes.

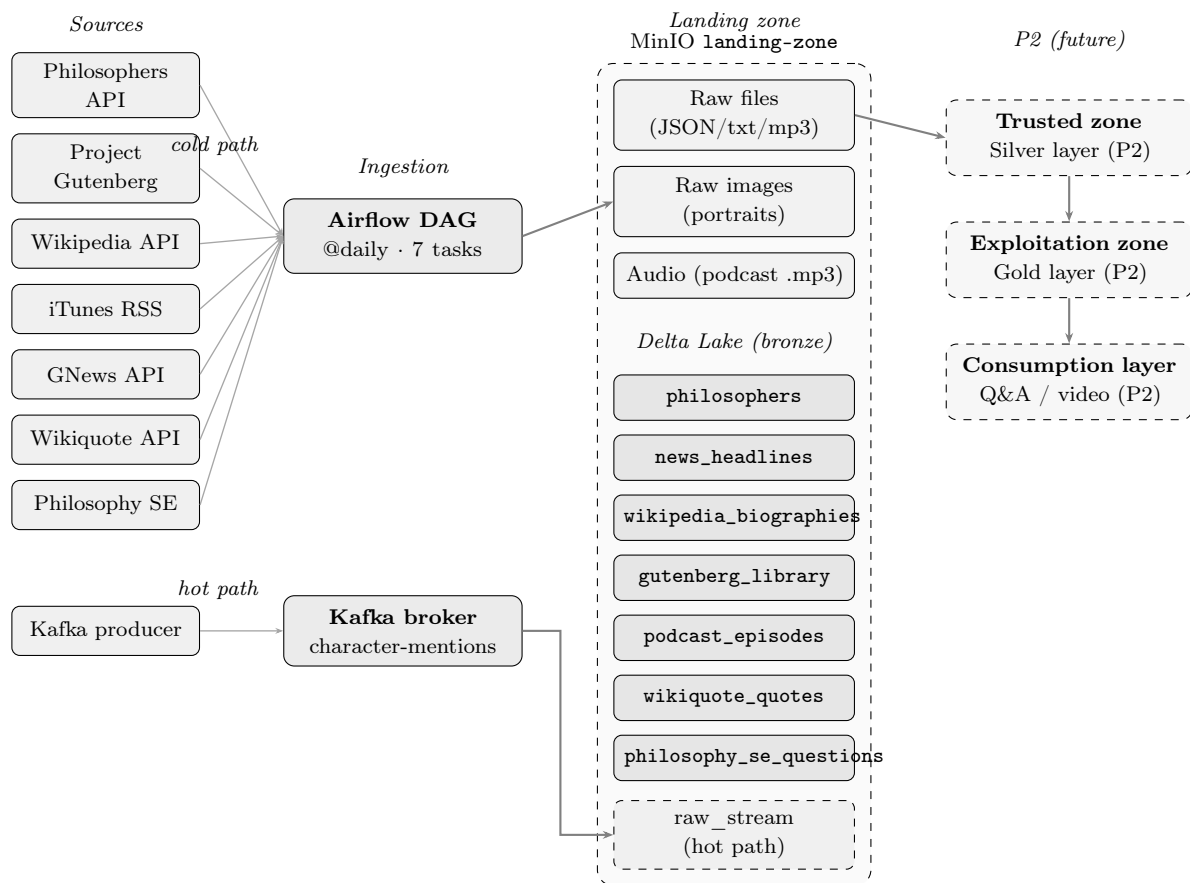


Figure 1: Full pipeline architecture — P1 scope (Bronze Layer) with P2 black boxes.

2.2 Infrastructure Stack

The entire infrastructure is defined in a single `docker-compose.yml` and spins up **8 containers**, each with a clearly scoped responsibility. At the storage layer, `minio` (MinIO latest) serves as the S3-compatible object store forming the Bronze Landing Zone, exposed on ports 9000 (S3 API) and 9001 (web console). A companion one-shot container, `minio-init` (MinIO MC client), runs at startup to idempotently bootstrap the `landing-zone` bucket and then exits — ensuring

the bucket always exists before any ingestion task fires.

The metadata and orchestration layer is composed of three containers. **postgres** (PostgreSQL 13) acts as the Airflow metadata database, replacing the default SQLite backend which produces file-level deadlocks under parallel task execution. **airflow-webserver** (Apache Airflow 2.9.0) provides the web UI for DAG monitoring, manual triggering, and log inspection on port 8080, while **airflow-scheduler** runs alongside it to detect scheduled triggers and dispatch tasks to the LocalExecutor.

The streaming layer consists of three additional containers. **zookeeper** (Confluent CP 7.x) handles Kafka coordination — managing broker registration and topic metadata. **kafka** (Confluent CP 7.5.0) is the distributed streaming broker serving the **character-mentions** hot-path topic on port 9092. Finally, **kafka-ui** (Provectus Kafka UI) exposes a web interface on port 8085 for real-time topic monitoring and consumer group health inspection, without requiring any CLI access to the containers.

Layer	Container	Technology	Role
Storage	minio	MinIO latest	S3-compatible object store — Landing Zone (:9000, :9001)
Storage	minio-init	MinIO MC client	One-shot bucket bootstrap; exits after creation
Orchestration	postgres	PostgreSQL 13	Airflow metadata DB; prevents parallel execution deadlocks
Orchestration	airflow-webserver	Airflow 2.9.0	DAG monitoring and log inspection (:8080)
Orchestration	airflow-scheduler	Airflow 2.9.0	Trigger detection and task dispatching
Streaming	zookeeper	Confluent CP 7.x	Kafka broker coordination and topic metadata
Streaming	kafka	Confluent CP 7.5.0	Hot-path streaming broker (:9092)
Streaming	kafka-ui	Provectus Kafka UI	Topic monitoring and consumer group health (:8085)

2.3 P2 Vision (Black Boxes)

While the Trusted and Exploitation Zones are not implemented in P1, their roles are already defined at a conceptual level. The **Trusted Zone (Silver Layer)** will clean and standardize all raw inputs — stripping Gutenberg boilerplate, normalizing schemas, transcribing podcast audio via Whisper ASR, and enforcing data quality checks. Finally, the **Exploitation Zone (Gold Layer)** will transform this clean data into AI-ready feature stores: embedding indexes over primary texts and Wikiquote quotes, per-figure knowledge graphs, aggregated news sentiment

signals, and fine-tuning datasets derived from the Philosophy Stack Exchange Q&A pairs. These curated features would then be consumed by a downstream generative application, whether a Q&A engine, a short-form video generator, or a full podcast pipeline.

3. Data Ingestion

3.1 Ingestion Strategy

Two ingestion strategies are deployed in parallel, each matched to the nature and urgency of its data source.

Cold path: Historical texts and biographical metadata do not change intraday, making a scheduled batch approach (@daily) the natural fit. Daily ingestion is sufficient to capture any source updates while keeping API usage well within free-tier quotas.

Hot path: Character mentions are time-sensitive social signals that benefit from being captured as they occur. A continuous streaming pipeline via Apache Kafka enables future real-time trend analysis without waiting for the next batch cycle. The streaming pipeline operates continuously and independently of the daily DAG schedule, as batch orchestration is architecturally incompatible with long-running consumer processes.

3.2 The Character Registry — Single Source of Truth

Before diving into individual scripts, the most important architectural element of the ingestion layer must be highlighted: the `character_registry.py` module. Every figure-specific ingestion script imports its target entities from this file — none hardcode any figure names, slugs, or search terms.

Design Pattern: Registry-Driven Ingestion

Adding a new historical figure to the entire pipeline — across all figure-specific data sources — requires a single dictionary append to `TARGET_FIGURES`. All scripts pick up the change automatically on the next run with zero code modifications.

Each registry entry carries source-specific search terms as separate fields:

```
{
  "domain":          "philosophy",
  "api_name":         "Immanuel Kant",      # Philosophers API exact match
  "guttenberg_search": "Kant",              # Gutendex keyword search
  "guttenberg_author_slug": "kant",         # Author filter & S3 path key
  "wikidata_label":   "Immanuel Kant",      # Wikipedia / Wikiquote title
}
```

The registry also exposes convenience helpers: `figures_by_domain('philosophy')` returns only philosophers (used by scripts targeting domain-specific APIs), while `TARGET_FIGURES` returns all nine figures (used by universal scripts like `wikipedia_ingest.py` and `wikiquote_ingest.py`). Note that `philosophyse_ingest.py` does not consume the registry — it ingests platform-level Q&A data independent of any specific figure.

3.3 Cold-Path Ingestion Scripts

3.3.1 `philosophers_ingest.py` — Structured Metadata & Portraits

This script fetches the complete catalog of 114 philosophers from the Philosophers API in a single HTTP request and filters records to only the target figures defined in the registry via a lowercase name lookup. It attaches academic deep-link URLs (Stanford SEP, IEP, Wikipedia) derived from the raw API fields, then uploads the filtered catalog as a single `philosophers_catalog.json`. It also iterates through the nested `images` dictionary (thumbnails, illustrations, face crops) and downloads every portrait image separately. An idempotency check (`head_object`) is applied per image before downloading, so already-present portraits are silently skipped.

Storage: `s3://landing-zone/philosophers_api/raw_images/{domain}/{slug}/{category}/{key}.jpg`

3.3.2 `guttenberg_ingest.py` — Primary Texts (Unstructured)

This script queries the Gutendex API using each figure's `guttenberg_search` term and applies a critical **author attribution filter**: it only retains books where the target figure actually appears as an *author* in the Gutendex response, preventing the ingestion of books that merely mention the figure. For each confirmed book, a `{slug}_catalog.json` provenance record is uploaded first, then the best available plain-text format is resolved in priority order (UTF-8 → ASCII → any plain content-type) and downloaded raw to MinIO. A mandatory 1.5-second delay is enforced between downloads to comply with Project Gutenberg's robot access policy, and idempotency is applied per book via `head_object` since public-domain texts are immutable.

Storage: `s3://landing-zone/guttenberg/raw_text/{domain}/{slug}_{book_id}_{title}.txt`

3.3.3 `wikipedia_ingest.py` — Universal Biographical Backbone

The Wikipedia REST API is called for every figure in `TARGET_FIGURES` regardless of domain. This universality is by design: Wikipedia provides factual grounding for figures that may have limited coverage in domain-specific sources. The script normalizes article titles (spaces → underscores) and includes a custom `User-Agent` header as required by Wikipedia's bot policy. Results are organized by domain subdirectory.

Storage: `s3://landing-zone/wikipedia/raw_json/{domain}/{slug}_wikipedia.json`

3.3.4 `podcast_audio_ingest.py` — Conversational Dynamics (Unstructured)

Rather than hardcoding specific podcast channels, this script implements a **discovery-based approach**: it queries the iTunes Search API for broad topics, discovers the highest-ranked channels by popularity, parses each channel's RSS feed to extract episode enclosures, and streams the binary audio files directly into MinIO using `upload_fileobj` — avoiding any local disk buffering. A JSON metadata envelope is generated per episode, decoupling the audio blob from its provenance.

Storage: `s3://landing-zone/podcasts/raw_audio/{podcast_slug}/ep_{id}.mp3`

3.3.5 `news_ingest.py` — Daily Trending Headlines

This script queries the GNews `/api/v4/top-headlines` endpoint (not a keyword search) for three categories: *world*, *technology*, and *science*. Each article is tagged with its `_source_category` field

before all results are aggregated into a single daily snapshot. Files are named `news_snapshot_YYYYMMDD.json` and are **safely overwritten** on repeated daily runs, ensuring the most recent articles always win while preventing data accumulation.

Storage: `s3://landing-zone/news_api/raw_json/news_snapshot_{YYYYMMDD}.json`

3.3.6 wikiquote_ingest.py — Verified Quotes

This script queries the Wikiquote MediaWiki API for every figure in `TARGET_FIGURES`, using the `wikidata_label` field as the page title with spaces replaced by underscores. For each figure, the full page content is retrieved and validated — the script checks for the presence of a valid page (rejecting MediaWiki -1 page IDs that indicate missing articles) before uploading. A `head_object` idempotency pre-check skips figures whose quotes are already stored, since verified quotation corpora are stable and rarely change. Results are organized by domain subdirectory, consistent with the Wikipedia ingestion pattern.

Storage: `s3://landing-zone/wikiquote/raw_json/{domain}/{slug}_wikiquote.json`

3.3.7 philosophyse_ingest.py — Q&A Format Examples

This script fetches the top 500 highest-voted questions from the Philosophy Stack Exchange via the Stack Exchange API, across five paginated requests of 100 results each. A one-second sleep between pages respects the API rate limit. Each question object is tagged with an `_ingested_at` timestamp before all results are aggregated into a single daily snapshot. The overwrite-by-date strategy (matching `news_ingest.py`) ensures the daily run always reflects the current top-voted content without accumulating redundant snapshots.

Storage: `s3://landing-zone/philosophy_se/raw_json/philosophy_se_snapshot_{YYYYMMDD}.json`

3.4 Hot-Path Ingestion — Apache Kafka Streaming

3.4.1 stream_producer.py — Event Generation

The Kafka producer generates synthetic real-time character mention events at random intervals (1–5 seconds), simulating a live social media feed. Each event carries: character name, domain, templated message body, sentiment score (uniform random in $[-1, 1]$), and source platform tag. It includes a retry loop with 5-second backoff to handle Kafka broker warm-up. The architecture of this producer is designed to be easily extensible; in a production deployment it would be replaced by a live social media API listener, capturing real-world discourse around the figures in real time.

3.4.2 stream_consumer.py — Buffer-and-Flush to MinIO

The consumer subscribes to the `character-mentions` topic with consumer group `landing-zone-capture` and accumulates messages in memory. When the buffer reaches 10 messages, it flushes the batch as a timestamped JSON file to MinIO's `hot_path/raw_stream/` prefix. On shutdown (e.g. `KeyboardInterrupt`), the consumer flushes any remaining buffered messages before closing, preventing data loss for partial batches. This balances write efficiency with latency.

Remaining limitation: While partial-buffer data loss on shutdown has been mitigated via a `finally-block` flush, the consumer does not yet implement a time-based flush. If the stream goes silent for an extended period, buffered messages will not be persisted until the next event

arrives or the process is terminated. A periodic 60-second timer flush will be added in P2.

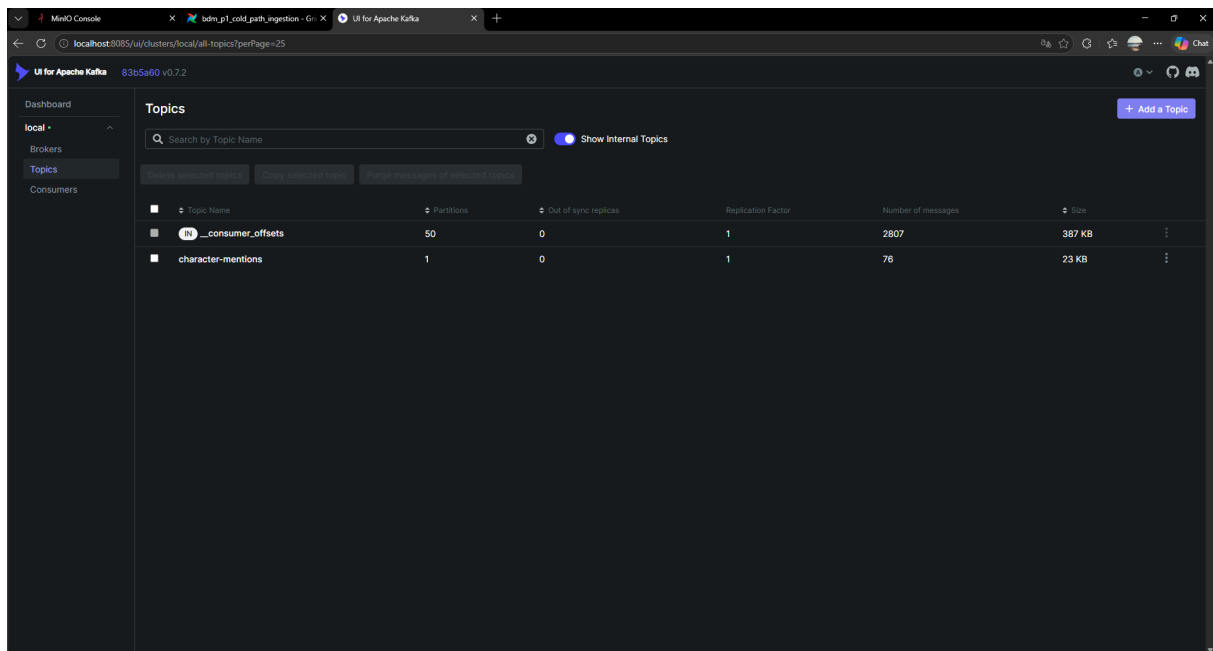


Figure 2: Kafka UI monitoring real-time streams on the `character-mentions` topic.

3.5 Idempotency Strategy Summary

Script	Strategy	Rationale
philosophers (images)	head_object skip	Portraits are static; re-downloading wastes API quota
gutenberg (texts)	head_object skip	Public-domain texts are immutable once published
news	Daily filename overwrite	Articles change intraday; today's snapshot should reflect the latest fetch
philosophers (catalog)	Always overwrite	Ensures the latest API truth is always present
podcast audio	head_object skip	Audio files are large; re-downloading is expensive
wikipedia	Always overwrite	Biographies can be edited; daily refresh is cheap
wikiquote	head_object skip	Verified quote corpora are stable and rarely change
philosophy SE	Daily filename overwrite	Top-voted questions shift over time; latest snapshot always wins

4. Landing Zone

4.1 Storage Paradigm: Data Lakehouse

The Landing Zone is implemented as a **Data Lakehouse** — combining the heterogeneous, schema-flexible storage of a Data Lake with the structured metadata management features of a Data Warehouse. This is achieved through two complementary technologies:

- **MinIO** — an S3-compatible local object store that handles the physical storage of all raw data objects (JSON, plain text, audio, images) and provides the raw-read layer.
- **Delta Lake** — an Open Table Format (OTF) built on top of MinIO’s Parquet storage that provides ACID transactions, schema enforcement, time travel (versioned reads), and automatic metadata management for all structured components of the Bronze layer.

Design Rationale

A plain file system would suffice for raw storage but would immediately produce a “data swamp” as the pipeline grows. Delta Lake’s transaction log ensures that any failed mid-write is automatically rolled back, preventing partial or corrupted tables from entering the pipeline. Time travel enables rolling back to any previous table version — a critical feature for AI training pipelines, where data quality regressions can silently poison downstream models.

The choice of MinIO over raw AWS S3 is also deliberate: `boto3` API calls are identical, meaning **the entire pipeline is cloud-portable with a single environment variable change**. Swapping `MINIO_ENDPOINT=localhost:9000` for an AWS endpoint is the only modification required to run this pipeline in production on AWS.

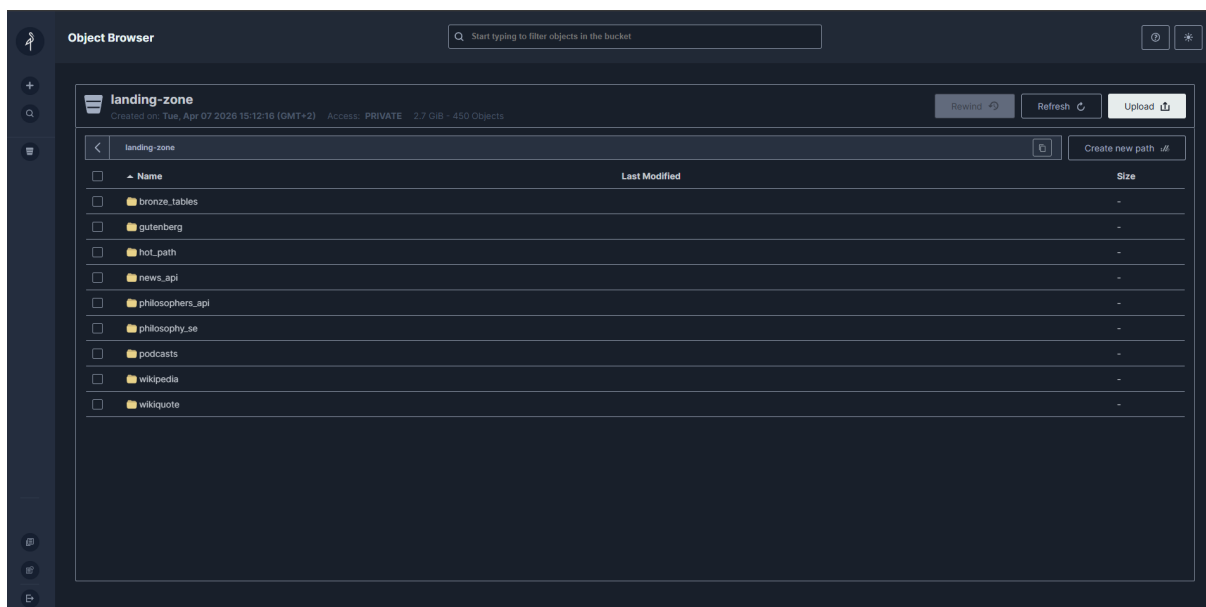


Figure 3: MinIO Console showcasing the Landing Zone bucket structure.

4.2 Data Organization

The `landing-zone` bucket is organized into eight top-level prefixes:

```
landing-zone/
|-- philosophers_api/
|   |-- raw_json/philosophy/philosophers_catalog.json
|   '-- raw_images/{domain}/{slug}/{category}/{key}.jpg
|-- gutenbergl/
|   '-- raw_text/{domain}/
|       |-- {slug}_catalog.json      # provenance metadata
|       '-- {slug}_{id}_{title}.txt  # raw book text
|-- wikipedia/
|   '-- raw_json/{domain}/{slug}_wikipedia.json
|-- wikiquote/
|   '-- raw_json/{domain}/{slug}_wikiquote.json
|-- philosophy_se/
|   '-- raw_json/philosophy_se_snapshot_{YYYYMMDD}.json
|-- podcasts/
|   |-- raw_audio/{podcast_slug}/ep_{id}.mp3
|   '-- metadata/{podcast_slug}/ep_{id}_meta.json
|-- news_api/
|   '-- raw_json/news_snapshot_{YYYYMMDD}.json
|-- hot_path/
|   '-- raw_stream/mentions_{timestamp}.json # hot path
'-- bronze_tables/                          # Delta Lake
    |-- philosophers/
    |-- news_headlines/
    |-- wikipedia_biographies/
    |-- gutenbergl_library/
    |-- podcast_episodes/
    |-- wikiquote_quotes/
    '-- philosophy_se_questions/
```

Three organizational principles are enforced consistently:

- **Domain-based subdirectory routing:** Files are organized by knowledge domain (`philosophy` / `science` / `literature`) within per-figure source prefixes, enabling domain-scoped queries without full-bucket scans.
- **Source-type separation:** Raw binary files (images, audio, text) are kept in dedicated `raw_*` prefixes, distinct from JSON metadata. This allows the Trusted Zone to selectively process only the formats it needs without iterating over binary blobs.
- **Standardized naming convention:** All files follow the pattern `{slug}_{identifier}_{descriptor}.{ext}`, making the authoring figure recoverable from the filename alone without parsing file contents.

4.3 Data Volume Summary

To substantiate the “Big Data” scope of the pipeline, the following table consolidates the data volumes produced by each source:

Source	Volume per Run	Growth Rate	Data Type
Gutenberg texts	~600 MB	Static (immutable)	Unstructured (plain text)
Podcast audio	~2 GB (40+ hrs)	Per discovery run	Unstructured (MP3 binary)
News snapshots	~150 KB/day	~4.5 MB/-month	Semi-structured (JSON)
Philosophy SE Q&A	~2 MB/snapshot	~60 MB/-month	Semi-structured (JSON)
Wikipedia bios	~100 KB	Static (daily overwrite)	Semi-structured (JSON)
Wikiquote quotes	~50 KB	Static (idempotent)	Semi-structured (JSON)
Philosophers API	~30 KB + images	Static (daily overwrite)	Semi-structured + images
Hot path stream	Variable	Continuous	Semi-structured (JSON)

The initial ingestion footprint exceeds **2.6 GB**, dominated by podcast audio and Gutenberg prose. The two append-mode Delta tables (`news_headlines` and `philosophy_se_questions`) ensure that data volume grows steadily over time, accumulating roughly **65 MB/month** of new structured content.

4.4 Delta Lake Bronze Tables

The `metadata_to_delta.py` script is the final task in the DAG and runs only after all seven ingestion tasks complete successfully. It reads the raw JSON files from MinIO, flattens them using `pandas.json_normalize`, and writes seven structured Delta tables to the `bronze_tables/` prefix:

Delta Table	Write Mode	Contents
<code>philosophers</code>	overwrite	Flattened Philosophers API records including all academic links and school metadata
<code>news_headlines</code>	append	Cumulative history of all daily snapshots — grows by ~30 rows/day across three categories
<code>wikipedia_bios</code>	overwrite	One row per figure across all domains with biography extract and structured metadata
<code>gutenberg_library</code>	overwrite	Catalog of every text file available per figure — an index for the raw text blobs
<code>podcast_episodes</code>	overwrite	Metadata index of all downloaded audio episodes with provenance envelope fields
<code>wikiquote_quotes</code>	overwrite	Verified quotes per figure extracted from Wikiquote — grounding layer for AI generation
<code>philosophy_se_questions</code>	append	Growing archive of top-voted philosophy Q&A pairs — direct training signal for output format

The two append-mode tables (`news_headlines` and `philosophy_se_questions`) build growing historical archives over time — a feature impossible with a simple daily overwrite. The `philosophy_se_questions` table in particular accumulates a richer training signal with each run as community voting patterns shift and new questions rise to the top. All other tables use overwrite mode since they represent current state rather than time series.

Delta Lake's Parquet storage provides automatic **column-level compression**, enabling far more efficient downstream queries than raw JSON. A query over `philosophy_se_questions` to retrieve all questions tagged with a specific topic can be resolved by reading only the relevant columns without deserializing the full question body — a property unique to columnar formats.

4.5 Orchestration

The entire cold-path pipeline is orchestrated by a single Apache Airflow DAG (`bdm\_p1\_cold\_path\_ingestion`) scheduled `@daily`, implementing a **fan-out / fan-in pattern**. A health-check gate runs first, seven ingestion tasks execute in parallel, and a final Delta conversion task fans them back in before the pipeline terminates.

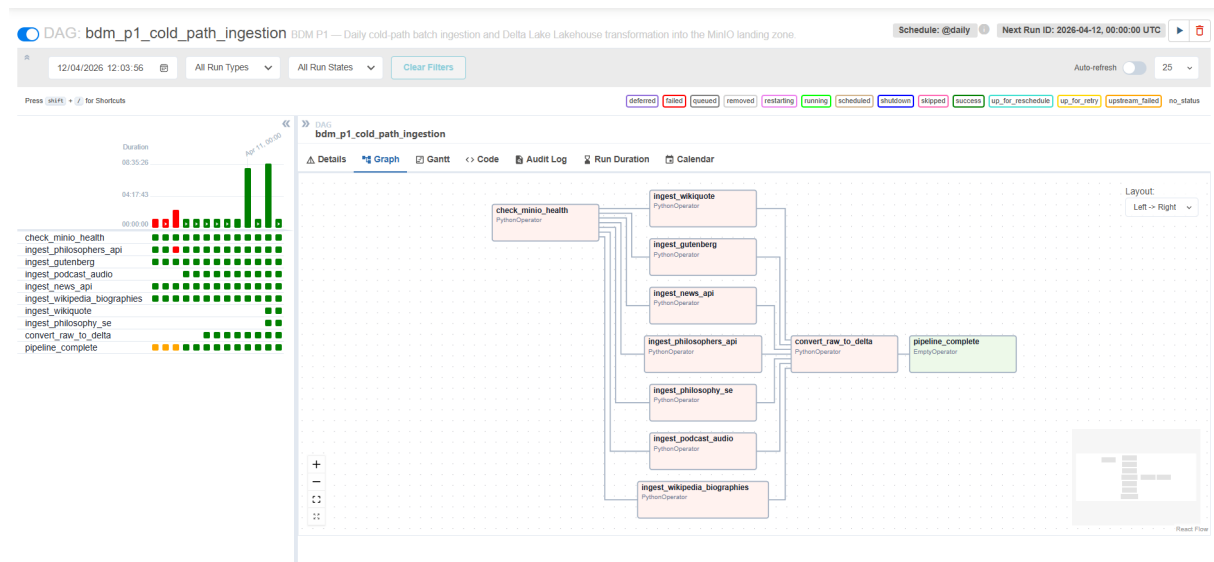


Figure 4: Airflow DAG graph view showing the 7-task fan-out / fan-in execution pattern.

The DAG is designed around several deliberate operational decisions. A **fail-fast health gate** (`check_minio_health`) verifies MinIO connectivity before any ingestion begins — if the object store is unreachable, the DAG aborts immediately rather than launching seven parallel tasks that would produce a partial, inconsistent load. Once the gate passes, all seven ingestion tasks run **concurrently**, reducing total wall-clock time from the sum to the maximum of individual runtimes. The Delta conversion task then waits for all seven to complete before running, guaranteeing that Bronze tables always reflect a full snapshot. Operationally, `max_active_runs=1` prevents concurrent runs from colliding on the same MinIO keys, `catchup=False` avoids exhausting the GNews daily quota through backfilling, and `retries=2` with a five-minute delay handles transient API failures gracefully.

4.6 Key Engineering Decisions

The character registry as a shared foreign key. The central design decision was to centralize all target entity configuration in a single `character_registry.py` module. Every figure-specific ingestion script imports its targets from this file, meaning that adding a new historical figure to the entire pipeline requires editing exactly one dictionary. More importantly, the registry establishes semantic coherence across sources: the shared `gutenberg_author_slug` field becomes the path key in MinIO, so files from Wikipedia, Gutenberg, Wikiquote, and the Philosophers API for the same figure all land under the same slug. This makes cross-source joins trivial in P2 without requiring any explicit foreign key management at ingestion time.

Subprocess-based task execution. Each Airflow task invokes its ingestion script as a subprocess via `sys.executable` rather than inlining the logic as a Python callable inside the DAG. The cost is a small process-spawning overhead per task. The gain is that every script remains independently executable: `python ingestion/wikipedia_ingest.py` runs identically outside of Airflow, making local testing and debugging fast without spinning up the full orchestration stack.

Delta Lake as a metadata index, not a full store. Unstructured content (600 MB of Gutenberg prose, 2 GB of podcast audio, portrait images) is stored as raw files in dedicated

MinIO prefixes. The Delta Lake Bronze tables store only the structured metadata: book catalogs, episode envelopes, biography extracts, verified quotes. This separation keeps storage efficient and avoids forcing binary blobs through Parquet serialization, while the shared slug conventions from the registry ensure that any Delta table row can be linked back to its corresponding raw files.

Differentiated idempotency per data type. Rather than applying a single blanket strategy, each script implements the idempotency model that fits its data. Static assets (images, books, audio, quotes) use a `head_object` pre-check and are skipped if already present. News and Stack Exchange snapshots use a daily filename overwrite, so the latest run always wins. The philosophers catalog always overwrites to guarantee the freshest API state. This avoids both redundant re-downloads of immutable content and stale snapshots of time-sensitive data.

5. Conclusions and Next Steps

5.1 Summary of P1 Achievements

This deliverable presents a fully operational Bronze Layer Data Lakehouse for the Historical Conversational AI system. The infrastructure runs as eight Docker containers launched with a single command, reaching a healthy state in under 90 seconds. Seven cold-path ingestion scripts cover all three required data categories across seven external sources, complemented by a Kafka-based hot-path streaming pipeline. The two new sources added in this iteration — Wikiquote and Philosophy Stack Exchange — directly address the AI system’s most critical output quality risks: quote hallucination and format misalignment. All structured outputs are stored as seven Delta Lake Bronze tables with ACID guarantees, time travel, and Parquet columnar storage. The pipeline is orchestrated by a daily Airflow DAG with a fail-fast health gate and parallel task execution, and the entire stack is cloud-portable with a single environment variable change.

5.2 Known Limitations and P2 Roadmap

Several limitations of the current implementation are acknowledged and earmarked for P2. The Kafka consumer currently flushes buffered messages on shutdown via a `finally`-block guard, preventing data loss for partial batches; however, it does not yet implement a time-based periodic flush, meaning buffered messages may be delayed indefinitely if the stream goes silent — a 60-second timer flush will be added in P2. Gutenberg texts retain their boilerplate headers and footers and are not yet ML-ready; the Trusted Zone will strip these using the well-defined `START/END PROJECT GUTENBERG` markers. No data quality validation currently exists between ingestion and the Delta tables: schema enforcement, null checks, deduplication, and a quarantine mechanism for failed records will be introduced in the Trusted Zone. Podcast audio is stored raw without transcription; the Exploitation Zone will run Whisper ASR over these files and store the resulting transcripts alongside the audio in Delta metadata. The `news_headlines` and `philosophy_se_questions` tables grow unboundedly; P2 will introduce Z-ORDER partitioning by date and a 90-day rolling retention window for both. The Stanford Encyclopedia of Philosophy URLs already stored in the `philosophers` Delta table represent an untapped high-quality source; a future `sep_ingest.py` ingestion script would retrieve the full structured article text per figure, providing topic-organized secondary analysis that would significantly improve RAG retrieval

precision. Finally, no pipeline monitoring is currently configured; Airflow email alerts and a Prometheus/Grafana stack are planned for the final deliverable.

5.3 Evaluation Against Project Criteria

The implementation satisfies the four core evaluation dimensions defined in the project statement. **Dynamicity** is achieved through the registry-driven pattern: a single append to `character_registry.py` integrates a new figure across all figure-specific data sources, and `figures_by_domain()` enables source-specific filtering without modifying any script. **Reusability** is ensured by keeping every ingestion script independently executable, with shared `boto3` client factories, upload helpers, and idempotency checks consistent across the codebase. **Openness** is guaranteed by MinIO's S3-compatible API — cloud migration requires changing one environment variable — and by Delta Lake's Parquet foundation, which is queryable by any modern engine including Spark, DuckDB, and Polars without Delta-specific knowledge. **Reproducibility** is provided by Docker Compose: a single `docker compose up -d` erects the full stack from a cold start, with version-pinned dependencies and idempotent bucket bootstrapping via `minio-init`.

The architecture establishes a **scalable, extensible, and production-faithful Bronze Layer** that provides a solid foundation for the P2 Trusted and Exploitation Zones. All raw data required to power a historically-grounded, current-events-aware conversational AI is now being collected, organized, and stored in a queryable lakehouse format — ready for the cleaning and enrichment steps to come.

A. Environment Setup Instructions

A.1 Prerequisites

- Docker Engine $\geq$ 24.x with Docker Compose plugin
- Python 3.10+ (for local testing outside Airflow)
- ~5 GB of free disk space
- A GNews API key — free tier at gnews.io (100 req/day)

A.2 Environment Setup

Create a `.env` file in the project root:

```
# MinIO Object Store
MINIO_ENDPOINT=localhost:9000
MINIO_ACCESS_KEY=admin
MINIO_SECRET_KEY=password
MINIO_BUCKET=landing-zone

# Kafka
KAFKA_BOOTSTRAP_SERVERS=localhost:9092

# External APIs
NEWS_API_KEY=YOUR_GNEWS_API_KEY_HERE
```

A.3 Launching the Full Stack

Step 1 — Start all containers:

```
docker compose up -d
```

This single command boots 8 containers: MinIO, MinIO-init (bucket bootstrap), PostgreSQL, Airflow Webserver, Airflow Scheduler, Zookeeper, Kafka, and Kafka UI. First boot takes approximately 90 seconds.

Step 2 — Verify health:

```
docker compose ps
```

All services except `minio-init` (which exits after creating the bucket) should show **healthy**.

A.4 UI Access

Service	URL	Credentials
Airflow Web UI	http://localhost:8080	admin / admin
MinIO Console	http://localhost:9001	admin / password
Kafka UI	http://localhost:8085	(no auth required)

A.5 Triggering the Pipeline

Via Airflow UI (recommended):

1. Navigate to `http://localhost:8080` and log in.
2. Locate the DAG `bdm_p1_cold_path_ingestion` and unpause it.
3. Click the **Run** button to trigger a manual execution.
4. Click the DAG name → *Graph View* to monitor real-time task execution.

Via CLI:

```
docker exec airflow-scheduler airflow dags trigger bdm_p1_cold_path_ingestion
```

A.6 Local Testing (without Airflow)

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
docker compose up -d          # MinIO + Kafka must be running

# Cold-path scripts
python ingestion/philosophers_ingest.py
python ingestion/gutenberg_ingest.py
python ingestion/wikipedia_ingest.py
python ingestion/wikiquote_ingest.py
python ingestion/philosophyse_ingest.py
python ingestion/news_ingest.py
python ingestion/podcast_audio_ingest.py
python ingestion/metadata_to_delta.py

# Hot-path (run in separate terminals)
python ingestion/stream_producer.py
python ingestion/stream_consumer.py
```